```
% 2-D FEM code, triangular elements
k = 300000;  % Constants
NL = 3; % Number of Local nodes
NG = 3; % Number of Gauss points
NN = length(xp); # nodes
NE = length(T(:,1));# elements

gps = [1 4 1] / 6; ξ
gpe = [1 1 4] / 6; η
gw = [1 1 1] / 6; gauss weights
f = zeros(NL,NG); φi shape fxns
dfds = zeros(NL,NG); dφi/dξ
dfde = zeros(NL,NG); dφi/dη
for g = 1:NG for each gauss point
    s = gps(g); } set values of ξ, η
    e = gpe(g); }
    f(1,g) = 1 - s - e;
    f(2,g) = s;          } set up basis fxns φi for Δ mesh
    f(3,g) = e;
    dfds(1,g) = -1;
    dfds(2,g) = 1;       } dφi/dξ
    dfds(3,g) = 0;
    dfde(1,g) = -1;
    dfde(2,g) = 0;       } dφi/dη
    dfde(3,g) = 1;
end

b = zeros(NN,1); load vector
A = zeros(NN,NN); stiffness
M = zeros(NN,NN); mass matrix, not used

for e = 1:NE elements
    for g = 1:NG gauss pts
        x = 0; } physical coordinates
        y = 0; } of the gauss point g
        dxds = 0; dx/dξ
        dxde = 0;     dx/dη
        dyds = 0;          } mapping reference
        dyde = 0;          } for physical space
                      dy/dξ
                 dy/dη
        for i = 1:NL# local nodes
            ig = T(e,i); ↗ pos. weight in x-direction
                         ↳ φi basis fxn
            x = x + xp(ig) * f(i,g);
            y = y + yp(ig) * f(i,g);
            dxds = dxds + xp(ig) * dfds(i,g);
            dxde = dxde + xp(ig) * dfde(i,g);
            dyds = dyds + yp(ig) * dfds(i,g);
            dyde = dyde + yp(ig) * dfde(i,g);
        end
        J = dxds * dyde - dxde * dyds; Jacobian
        for il = 1:NL local nodes
            ig = T(e,il); set position globally
            dfidx = (dyde * dfds(il,g) - dyds * dfde(il,g)) / J; dφi/dx
            dfidy = (dxds * dfde(il,g) - dxde * dfds(il,g)) / J; dφi/dy
            b(ig,1) = b(ig,1) - k * f(il,g) * gw(g) * J;
            ↳ load vector   bi = ∫ k φi dA
```

J = $\begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{bmatrix}$ area scaling

```
                                local trial fxns
            for jl = 1:NL  local nodes for each of the unknown ψ_j
                jg = T(e,jl); set global position
                dfjdx = (dyde * dfds(jl,g) - dyds * dfde(jl,g)) / J; dψj/dx
                dfjdy = (dxds * dfde(jl,g) - dxde * dfds(jl,g)) / J; dψj/dy
                A(ig,jg) = A(ig,jg) - dfidx * dfjdx * gw(g) * J; Building stiffness matrix
                A(ig,jg) = A(ig,jg) - dfidy * dfjdy * gw(g) * J; Building stiffness matrix
            end
        end                  A_ij = ∫ ∇ψ_i · ∇ψ_j dA = ∫ [∂/∂x; ∂/∂y] ψ_i · [∂/∂x; ∂/∂y] ψ_j
    end
end


for i = 1:NN loop through all nodes
    if (Dirichlet(i,1) > 0) if the node is part of the Dirichlet boundary (as defined prev.)
        A(i,:) = 0; set the row to 0
        A(i,i) = 1; set diagonal to 1
        b(i,1) = 0; set RHS side boundary val (in this case it's 0)
    end
end

u = A\b; solve for unknowns

Q = 0;
for e = 1:NE
    for g = 1:NG
        x = 0;
        y = 0;
        dxds = 0;
        dxde = 0;
        dyds = 0;
        dyde = 0;
        for i = 1:NL
            ig = T(e,i);
            x = x + xp(ig) * f(i,g);
            y = y + yp(ig) * f(i,g);
dx/ds       dxds = dxds + xp(ig) * dfds(i,g);
dx/dη       dxde = dxde + xp(ig) * dfde(i,g);
dy/ds       dyds = dyds + yp(ig) * dfds(i,g);
dy/dη       dyde = dyde + yp(ig) * dfde(i,g);
        end
        J = dxds * dyde - dxde * dyds;
        for il = 1:NL
            ig = T(e,il);
            Q = Q + u(ig) * f(il,g) * gw(g) * J;
        end
    end
end
```

same mapping as before

$Q = \int_A u(x,y)\, dA$

integrate over u